Table-driven Scanners

Scanners

- The **Scanner's** task is to transform a stream of characters into a stream of words in the input language. Each word must be classified into a syntactic category, or "part of speech."
- The scanner is the only pass in the compiler to touch every character in the input program.
- It groups characters together to form words and punctuation in the source language, they lend themselves to fast implementations.
- Compiler writers place a premium on speed in scanning, in part because the scanner's input is larger, in some measure, than that of any other pass, and, in part, because highly efficient techniques are easy to understand and to implement.

Cycle of Constructions

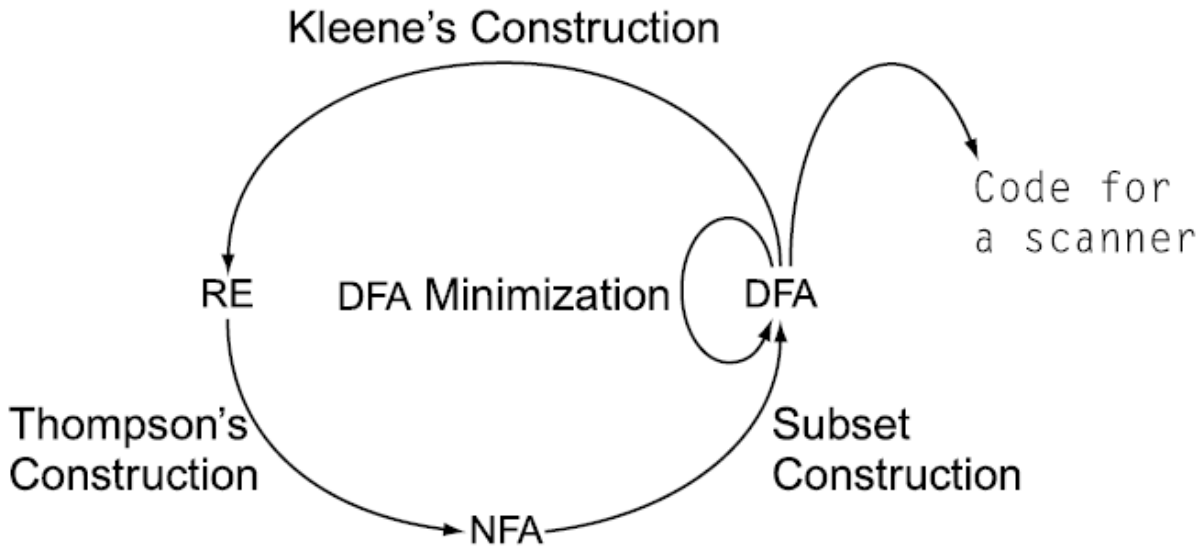


FIGURE 2.3 The Cycle of Constructions.

Scanning

- **Scanning** is the first stage of a three-part process that the compiler uses to understand the input program.
- The scanner, or lexical analyzer, reads a stream of characters and produces a stream of words.
- It aggregates characters to form words and applies a set of rules to determine whether or not each word is legal in the source language.
- If the word is valid, the scanner assigns it a syntactic category, or part of speech.
- Automatic tools for scanner generation are common. These tools process a mathematical description of the language's lexical syntax and produce a fast recognizer.
- Alternatively, many compilers use hand-crafted scanners; because the task is simple, such scanners can be fast and robust.

Scanning (cont..)

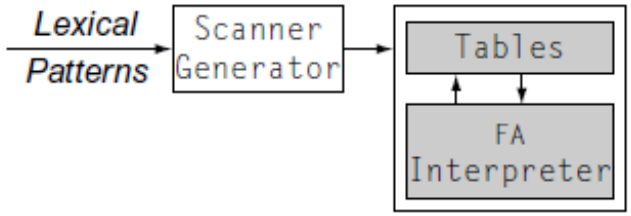
- **Recognizer**
  - a program that identifies specific words in a stream of characters
- **Syntactic category**
  - a classification of words according to their grammatical usage
- **Microsyntax**
  - The lexical structure of a language

Types of Scanners

- **Three implementation strategies** for converting a DFA into executable code:
  - a table-driven scanner,
  - a direct-coded scanner, and
  - A hand-coded scanner.
- All of these scanners operate in the same manner, by simulating the DFA. T
- They repeatedly read the next character in the input and simulate the DFA transition caused by that character.
- This process stops when the DFA recognizes a word.

Table-driven scanner

- The **table-driven approach** uses a skeleton scanner for control and a set of generated tables that encode language-specific knowledge.



■ **FIGURE 2.13** Generating a Table-Driven Scanner.

- The compiler writer provides a set of lexical patterns, specified as regular expressions. The scanner generator then produces tables that drive the skeleton scanner.

Table-driven scanner

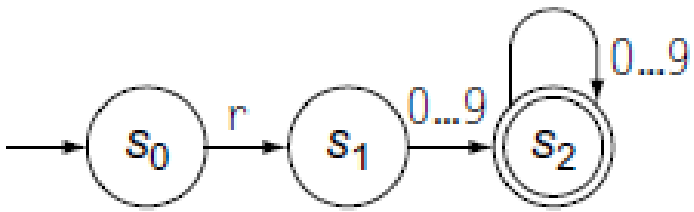
- The idea of a table-driven scanner is simple -- we have a table that says what to do if we are in one particular state and see a particular input character. This is basically the transition table for the DFA.
- Building the table can be a pain, but then your scanner is a simple loop. The table size can be significant (Suppose a Scanner for BPL (Basic Programming Language) has 35 tokens, so the DFA needs at least 35 states; there are at least 128 possible input characters, so you need a table with about 5,000 entries for BPL).
- But, for a full implementation of C or Java it wouldn't be too big to reside in memory.

Types of Tables in Scanner

- Uses Stack algorithm driven by three tables to simulate the DFA
- **Input Classification Table:**
  - Given a char, identifies what kind of character it is relevant to our DFA.
  - Example: a, digit, lowercase alpha, whitespace etc.
- **Transition Table:**
  - Given the current state and the classification for the current input character, identifies the correct next state.
- **Token Type Table:**
  - Given the ending state, identifies what kind of token that represents (or invalid: no token recognized)

A Table-Driven Scanner for Register Names

RE: $r\,[0 \ldots 9]^+$



The Underlying DFA

r	0, 1, 2, ..., 9	EOF	**Other**
Register	*Digit*	*Other*	*Other*

The Classifier Table, `CharCat`

	Register	***Digit***	***Other***
s_0	s_1	s_e	s_e
s_1	s_e	s_2	s_e
s_2	s_e	s_2	s_e
s_e	s_e	s_e	s_e

The Transition Table, δ

s_0	s_1	s_2	s_e
invalid	*invalid*	*register*	*invalid*

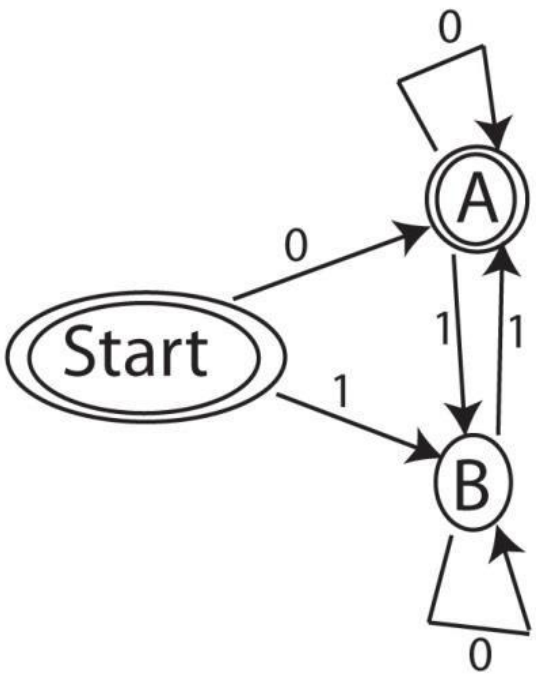
The Token Type Table, `Type`

Algorithm (RE for ILOC register names)

```
NextWord()
  state ← s0 ;
  lexeme ← " ";
  clear stack;
  push(bad);

  while (state≠se) do
    NextChar(char);
    lexeme ← lexeme + char;

    if state ∈ SA
        then clear stack;

    push(state);

    cat ← CharCat[char];
    state ← δ[state,cat];
  end;

  while(state ∉ SA and
        state≠bad) do
    state ← pop();
    truncate lexeme;
    RollBack();
  end;

  if state ∈ SA
    then return Type[state];
    else return invalid;
```

Example: Table driven Scanner

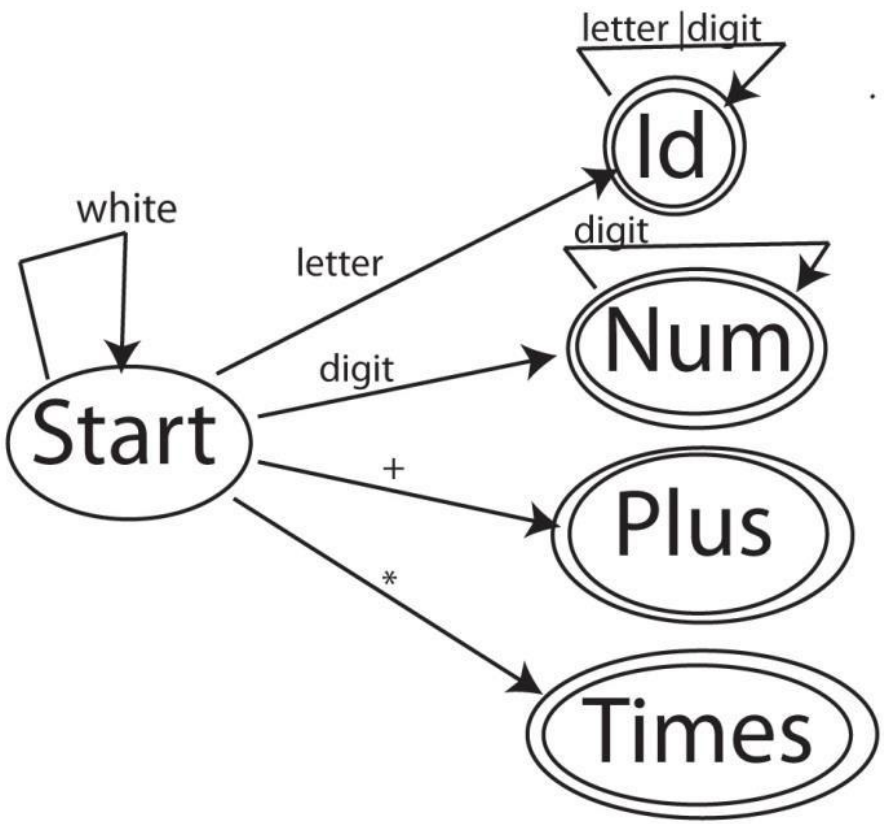
- Here is a simple example:
  - a table that represents a DFA:



	Char		
State	0	1	End-Of-String
Start	A	B	Accept
A	A	B	Accept
B	B	A	Reject

Example: Table driven Scanner

- Things get a little more complex if you have an actual scanner, which has to find token boundaries and emit tokens.
- Consider the following DFA, that represents a scanner that finds Id, Number, + and * tokens:



Example: Table driven Scanner

- We could represent this with the following table:

	Char					
State	letter	digit	+	*	white	eof
Start	Id	Num	Plus	Times	Start	T_ERR
Id	Id	Id	T_ID	T_ID	T_ID	T_ID
Num	T_NUM	NUM	T_NUM	T_NUM	T_NUM	T_NUM
Plus	T_PLUS	T_PLUS	T_PLUS	T_PLUS	T_PLUS	T_PLUS
Times	T_TIMES	T_TIMES	T_TIMES	T_TIMES	T_TIMES	T_TIMES

- Here a table entry Id means to go to the Id state and the next character of the input, while an entry such as T_ID means to issue a T_ID token and to not consume the current input character.
- Some people separate this into two tables: a Transition table for states, and an Action table that issues tokens and consumes input.